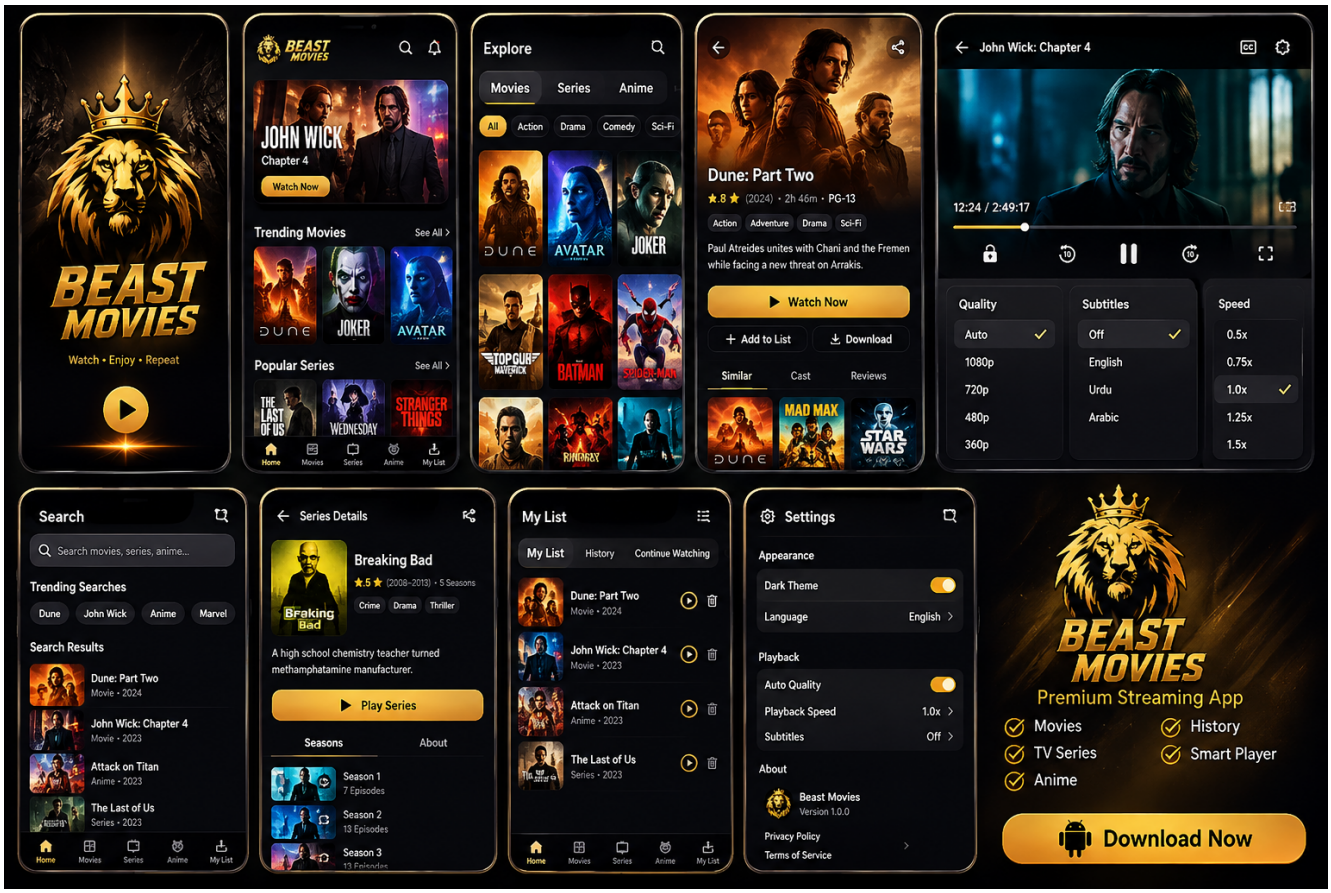
BEAST MOVIES

PRODUCT REQUIREMENTS DOCUMENT

Premium Android streaming application
Movies • TV Series • Anime



Version 1.0 | 1 November 2026 | Developer: Liaquat Ali | Package: com.beastmovies.app

Version: 1.0

Date: 1 November 2026

Product: Beast Movies

Platform: Native Android application (APK)

Package ID: `com.beastmovies.app`

Developer / Product owner: Liaquat Ali

Reference: User-supplied `Filmy Flow_1.9.apk` and supplied API request sample

Status: Product and UI specification; API schemas still require confirmation

---

1. Product summary

Beast Movies is a premium, API-driven Android media discovery and streaming application for Movies, TV Series and Anime. The supplied Filmy Flow APK is used as a visual reference for a black cinematic interface, gold/yellow brand accents, white text, poster-led browsing and a dedicated playback experience. Beast Movies must have its own original branding, assets and implementation.

The application must be a real working product, not a static prototype. Content, metadata, availability, playback sources and user entitlements must come from an API that the product owner is authorized to use. Do not fabricate content or stream URLs.

Product goals

• Make it easy to discover Movies, Series and Anime.

• Present rich details, seasons and episodes.

• Play supported authorized video streams reliably.

• Resume playback and maintain a useful watch history.

• Provide a polished, responsive experience on a range of Android devices.

• Keep API credentials and user data secure.

Non-goals for v1

• Building a new content delivery network.

• Bypassing DRM, paywalls, regional restrictions or provider controls.

• Scraping protected services or extracting private endpoints without permission.

• Building an admin backend unless the API owner supplies an admin API and explicitly scopes it.

• Offline downloads unless the API, rights and technical delivery explicitly permit them.

---

2. Reference APK inspection notes

The uploaded APK was inspected at the package/resource level. Visible bundled branding assets show a black background with a yellow/gold and white filmstrip/play emblem. Strings in the application binary indicate concepts such as Home, Movies, Series, Anime, Search, Details and Player, and the package includes Media3/ExoPlayer-related components.

The supplied APK also contains embedded service credentials/secrets in its binary. Treat these as compromised if they are real: revoke/rotate them and remove secrets from future builds. Do not copy embedded secrets into Beast Movies.

Important distinction: the generated UI images in this package are design concepts inspired by the reference's visual direction. They are not screenshots captured from the running reference APK and do not prove backend behavior.

---

3. Users and primary journeys

Primary user

A mobile viewer who wants to browse films, series and anime, inspect details, and play authorized streams on Android.

Core journeys

Discover and play a movie

• Launch app.

• Load Home sections from API.

• Open a movie card.

• Read details and available metadata.

• Tap Play.

• Request playback information from API.

• Start supported stream in Media3.

• Save playback progress and restore it later.

Watch a series episode

• Open Series or Anime.

• Open a title.

• Select a season.

• Select an episode.

• Load episode playback metadata.

• Play, save progress and optionally advance to the next episode if API data supports it.

Find and save content

• Open Search.

• Enter a query.

• Debounce input and call API.

• Open a result.

• Add it to My List.

• See it in the saved list.

---

4. Information architecture and navigation

Primary bottom navigation

• **Home** — featured content, trending, latest, recommendations and continue watching.

• **Movies** — movie catalogue, genres and filters.

• **Series** — TV series catalogue, seasons and episodes.

• **Anime** — anime catalogue and available language/audio variants.

• **My List** — watchlist, history and continue watching.

Search should be reachable from the Home/top app bar and from a dedicated search screen. Profile/settings are accessible from a profile or overflow action. Navigation must preserve appropriate scroll and filter state when users switch tabs.

Screen inventory

• Splash / launch

• Optional onboarding

• Home

• Movies catalogue

• Series catalogue

• Anime catalogue

• Search

• Search results

• Genre/category results

10. Movie details

11. Series details

12. Anime details

13. Season and episode selector

14. Video player

15. My List

16. Watch history

17. Continue watching

18. Profile / account (only if API supports account features)

19. Settings

20. About, privacy policy and terms

Authentication must be driven by the real API contract. If the API supports guest use, allow guest mode; do not invent a local account system that cannot sync with the service.

---

5. Visual design requirements

Visual direction

• Overall: premium cinematic dark interface.

• Background: near-black (`#08090D`).

• Elevated surfaces: charcoal (`#14151B`).

• Primary accent: warm gold (`#D6AC58`) with restrained brighter gold for active controls.

• Main text: white (`#FFFFFF`).

• Secondary text: muted grey (`#A1A1AA`).

• Cards: rounded 12–18dp, poster-first.

• Buttons: clear contrast, touch target at least 48dp.

• Navigation: dark bottom bar; gold for selected item.

• Typography: readable sans-serif with clear title/body hierarchy.

Layout behavior

• Use edge-to-edge layout carefully with status/navigation bar insets.

• Support portrait browsing and landscape playback.

• Use adaptive grids based on available width.

• Avoid text clipping, poster distortion and excessive nested scrolling.

• Respect Android font scaling and accessibility contrast.

• Provide loading skeletons, empty states, offline states and retry actions.

Home screen

• Brand/title and search action in top bar.

• Large featured hero card with backdrop, title and Play/Details actions.

• Horizontal shelves: Trending Movies, Popular Movies, Latest, Popular Series, Trending Anime and Continue Watching, depending on API sections.

• Each shelf is hidden or shows a useful empty state when the API has no data; do not display fake entries.

Catalogue screens

• Tabs or chips for relevant types and genres.

• Sort/filter controls only for fields supported by API.

• Lazy grid/list and pagination/infinite loading.

• Preserve selected filters and scroll position.

Details screens

• Backdrop, poster, title, release year, rating and runtime when provided.

• Synopsis, genres, cast/crew and language only when returned by API.

• Primary Play action only when playback is available.

• Add/remove My List action.

• Similar/recommended content only when API supports it.

• For Series/Anime: season selector and episode list.

Player screen

• Use Media3 controls with app-owned visual styling.

• Show/hide controls on tap.

• Play/pause, seek, timeline, fullscreen and orientation.

• Playback speed choices: 0.5x, 0.75x, 1.0x, 1.25x, 1.5x and 2.0x where supported.

• Quality selector only when multiple renditions or adaptive stream data are supplied.

• Subtitle and audio selectors only when tracks are available.

• Optional next episode only when episode ordering is supplied.

• Handle network changes, expired URLs and recoverable playback errors.

• Never promise a quality that the device/source cannot decode.

Settings

• Theme preference (dark by default).

• App language if translations exist.

• Default playback speed.

• Autoplay next episode toggle if supported.

• Clear cache action with confirmation.

• About, privacy policy and terms links.

• Version/build information.

---

6. Functional requirements

ID | Requirement | Priority

FR-001 | Load Home sections from the API | P0

FR-002 | Browse Movies with pagination | P0

FR-003 | Browse Series with pagination | P0

FR-004 | Browse Anime with pagination | P0

FR-005 | Open details using stable content IDs | P0

FR-006 | Request authorized playback metadata | P0

FR-007 | Play supported streams using Media3 | P0

FR-008 | Search content through API | P0

FR-009 | Show loading, empty and error states | P0

FR-010 | Save and restore playback progress | P1

FR-011 | Maintain watchlist | P1, if API/local policy supports

FR-012 | Show seasons and episodes | P1

FR-013 | Select available subtitle/audio tracks | P1

FR-014 | Continue watching | P1

FR-015 | Recommendations/similar content | P2, if API supports

FR-016 | Notifications for new releases | P2, if API supports

FR-017 | Offline downloads | Out of v1 unless licensed and API-supported

Data integrity

• Use stable IDs from the API, never array positions as IDs.

• Null or missing metadata must be handled safely.

• Unknown content types must not crash the app.

• Empty lists must render an empty state.

• Never infer a playable URL from a poster/detail URL.

• Do not persist expiring playback URLs as long-lived records.

---

7. API integration specification

Known supplied request

The user supplied this request sample:

• Method: `GET`

• Host: `h5-api.aoneroom.com`

• Path: `/wefeed-h5api-bff/home`

• Query: `host=movie-box.co`

• Protocol: HTTPS

• Response: JSON

• Request includes headers for origin/referer, language, client metadata and bearer authentication.

The API base URL inferred from that supplied request is:

```
https://h5-api.aoneroom.com/
```

The home response JSON has not yet been supplied. Therefore exact DTO fields, response envelope, error format, pagination format and authentication lifecycle are not confirmed. Do not implement guessed response models as production truth.

API work required before implementation

• Obtain permission and confirm the API's approved use, rate limits and client requirements.

• Obtain the complete home JSON response with tokens and personal identifiers redacted.

• Obtain endpoint documentation or authorized examples for catalogue, detail, search, season, episode and playback operations.

• Confirm required headers and which are dynamic versus fixed.

• Confirm whether tokens are per-user, device-bound or short-lived.

• Confirm pagination and sorting semantics.

• Confirm playback formats, URL expiry, referer requirements, DRM and subtitle/audio metadata.

• Confirm watchlist/history sync endpoints or decide whether those are local-only.

API client design

• Retrofit + OkHttp.

• Kotlin Serialization DTOs.

• Centralized interceptors for approved headers and auth.

• Explicit connect/read/write timeouts.

• Safe retry policy for idempotent requests only.

• Structured API error mapping.

• Cancellation support.

• Logging disabled or redacted in release builds.

• No bearer token or private key embedded in source, resources, APK or Git history.

• Use HTTPS and normal certificate validation; never install a trust-all SSL manager.

Candidate domain operations (names are internal abstractions, not asserted endpoint paths)

• `getHome()`

• `getMovies(page, filters)`

• `getSeries(page, filters)`

• `getAnime(page, filters)`

• `search(query, filters)`

• `getContentDetails(contentId)`

• `getSeasons(seriesId)`

• `getEpisodes(seasonId)`

• `getPlayback(contentId, episodeId?)`

• `saveWatchProgress(...)`

• `getWatchHistory()`

• `getWatchlist()`

• `addToWatchlist(contentId)`

• `removeFromWatchlist(contentId)`

The repository maps these operations to the actual approved API paths after documentation is received.

Proposed normalized domain models

`MediaItem`: id, title, type, synopsis, posterUrl, backdropUrl, releaseDate/year, rating, genres, runtime, language, availability.

`Season`: id, seriesId, seasonNumber, title, episodeCount.

`Episode`: id, seasonId, episodeNumber, title, synopsis, thumbnailUrl, duration.

`PlaybackInfo`: source list, MIME/format, expiry, headers permitted by provider, DRM configuration if applicable, subtitle tracks and audio metadata.

`PlaybackSource`: URL, MIME type, quality label, bitrate and codecs when supplied.

`SubtitleTrack`: language, label, URI, MIME/format, selection metadata.

All fields should be nullable or defaulted where the real API allows omissions.

---

8. Technical architecture

Stack

• Kotlin

• Jetpack Compose + Material 3

• MVVM + Clean Architecture

• Coroutines + StateFlow

• Hilt

• Retrofit + OkHttp

• Kotlin Serialization

• Room

• DataStore

• Navigation Compose

• Coil 3

• AndroidX Media3 ExoPlayer (core, UI, HLS and DASH modules as required)

• WorkManager only for justified deferrable work

• JUnit, MockK and Compose UI tests

Layer responsibilities

Presentation: Compose screens, ViewModels, UI state and user events.

Domain: use cases, domain models and repository contracts.

Data: API DTOs, mappers, local entities, repository implementations and cache policy.

Core: network setup, database, navigation, design system, logging, error handling and shared utilities.

Suggested project structure

BeastMovies/

■■■ app/

■ ■■■ src/

■ ■■■ main/

■ ■ ■■■ AndroidManifest.xml

■ ■ ■■■ java/com/beastmovies/app/

■ ■ ■■■ BeastMoviesApp.kt

■ ■ ■■■ MainActivity.kt

■ ■ ■■■ core/

■ ■ ■ ■■■ network/

■ ■ ■ ■■■ database/

■ ■ ■ ■■■ datastore/

■ ■ ■ ■■■ navigation/

■ ■ ■ ■■■ designsystem/

■ ■ ■ ■■■ errors/

■ ■ ■ ■■■ utils/

■ ■ ■■■ data/

■ ■ ■ ■■■ remote/dto/

■ ■ ■ ■■■ remote/mapper/

■ ■ ■ ■■■ local/

■ ■ ■ ■■■ repository/

■ ■ ■■■ domain/

■ ■ ■ ■■■ model/

■ ■ ■ ■■■ repository/

■ ■ ■ ■■■ usecase/

■ ■ ■■■ feature/

■ ■ ■ ■■■ home/

■ ■ ■ ■■■ movies/

■ ■ ■ ■■■ series/

■ ■ ■ ■■■ anime/

■ ■ ■ ■■■ details/

■ ■ ■ ■■■ search/

■ ■ ■ ■■■ player/

■ ■ ■ ■■■ watchlist/

■ ■ ■ ■■■ history/

■ ■ ■ ■■■ auth/

■ ■ ■ ■■■ settings/

■ ■ ■■■ di/

■ ■ ■■■ worker/

■ ■■■ test/

■ ■■■ androidTest/

■■■ build.gradle.kts

■■■ settings.gradle.kts

■■■ gradle.properties

■■■ README.md

Build configuration

• Minimum SDK: 24 (subject to required Media3/device support).

• Target/compile SDK: current stable version at implementation time.

• Package: `com.beastmovies.app`.

• Enable R8 for release after verifying keep rules.

• Produce debug and signed release APKs.

• Test 32-bit and 64-bit ABIs where dependencies and target devices permit; do not claim ABI support without a successful build/test.

---

9. Player engineering requirements

• Use Media3 `ExoPlayer` and `PlayerView` or Compose-compatible integration.

• Create a lifecycle-aware player owner/controller.

• Release resources when playback is finished or screen is disposed.

• Handle audio focus and noisy audio route events.

• Persist progress at sensible intervals and on pause/stop.

• Restore position after process recreation when content is still available.

• Use adaptive HLS/DASH selection where supplied by the stream.

• Add only supported media modules.

• Handle 401/403/404, expired URL, timeout, unsupported codec, decoder and network errors distinctly.

• Refresh an expired playback URL only through the provider's documented authorized flow.

• Do not automatically loop/restart the entire movie after a recoverable interruption.

• Use device capabilities and source metadata when presenting quality options.

• DRM must use the provider's authorized Media3 DRM configuration; never bypass DRM.

---

10. Local data and privacy

Room

Potential entities:

• WatchProgressEntity

• WatchlistEntity

• RecentSearchEntity

• CachedMediaMetadataEntity

Store only the minimum data needed. Do not cache private playback tokens as ordinary database fields.

DataStore

• Theme preference

• App language

• Playback speed

• Autoplay preference

• Other non-sensitive settings

Privacy

• Provide a privacy policy before public release.

• Explain what account/history data is sent to the API.

• Provide logout and local-data clearing where relevant.

• Avoid collecting unnecessary device identifiers.

• Redact credentials, tokens and personal identifiers from logs and crash reports.

---

11. Non-functional requirements

Area | Requirement

Reliability | No crash on missing/invalid API fields

Performance | Lazy lists, pagination and image caching

Accessibility | Content descriptions and scalable text

Responsiveness | Layout adapts to phone/tablet widths

Network | Clear loading, offline and retry states

Security | HTTPS, secure token storage and redacted logs

Maintainability | Feature modules/layers with tests

Battery | No unnecessary polling or permanent services

Playback | Proper player lifecycle and error recovery

---

12. Testing plan

Unit tests

• DTO parsing with real redacted API fixtures.

• DTO-to-domain mapping.

• Repository success/error handling.

• Pagination and duplicate prevention.

• ViewModel loading/success/empty/error states.

• Watch progress calculations.

• Playback URL refresh decision logic.

Integration tests

• Home API response.

• Movie/Series/Anime lists.

• Details and episode mapping.

• Search.

• Playback metadata parsing.

• Auth expiry and unauthorized response.

• History/watchlist if supported.

UI tests

• Bottom navigation.

• Search and filters.

• Open details and return.

• Add/remove My List.

• Season/episode selection.

• Player controls and orientation.

• Loading, empty and error states.

Device matrix

Test at minimum:

• Android API 24+ supported devices.

• Low-memory device (2–4 GB RAM).

• Modern Android device.

• Portrait browsing and landscape playback.

• Slow/interrupted network.

• Different screen sizes and font scaling.

---

13. Milestones

M0 — API contract discovery

Receive sanitized JSON fixtures and approved endpoint documentation. No production DTO assumptions before this.

M1 — Project foundation

Gradle, Compose, navigation, design system, Hilt, Retrofit and error handling.

M2 — Browse experience

Home, Movies, Series, Anime, Search, Details and pagination.

M3 — Player

Media3, playback metadata, quality, subtitles, audio tracks, progress and recovery.

M4 — Personal features

Watchlist, history, continue watching, settings and authentication if supported.

M5 — Quality

Tests, accessibility, performance, privacy and security review.

M6 — Release

Signed APK, release notes, build instructions and verification.

---

14. Acceptance criteria

The v1 release is accepted only when:

• The app builds from a clean checkout.

• The APK installs and launches on supported Android devices.

• Home content comes from the supplied authorized API.

• Movies, Series and Anime use real API data.

• Search returns real API results.

• Details screens use stable API IDs.

• Playback starts only from a valid authorized source.

• Playback errors are understandable and recoverable where possible.

• Progress resumes correctly when the API/local design supports it.

10. No primary button is decorative or inert.

11. Empty/loading/error states work.

12. Tests pass and release credentials are not embedded in the APK.

13. Privacy policy and third-party content rights are reviewed before distribution.

---

15. Open questions / blockers

The supplied home request is not enough to finish API integration. Required next inputs:

• Full home JSON response (redacted).

• Movie list and movie detail examples.

• Series, season and episode examples.

• Anime examples.

• Search response.

• Playback response with stream URLs redacted if necessary, but preserve field names and MIME/format metadata.

• Authentication/token lifecycle documentation.

• Confirmation of allowed client origins/headers, rate limits and playback rights.

Do not publish or hardcode the bearer token previously pasted in chat. Rotate it if it was a live credential.

---

16. Deliverables

• Complete Android Studio project.

• Beast Movies design system and original app branding.

• Functional API integration.

• Working Media3 player for authorized supported streams.

• Unit/integration/UI tests.

• Debug APK and signed release APK.

• README with build/configuration instructions.

• Privacy/security notes.

• UI reference image pack in `UI_Reference/`.